대화하며 코딩하는,
바이브코딩 첫걸음

AI와 함께 만드는 나만의 전공 웹 프로젝트

1차시

LLM(Large Language Model)이란?

- 방대한 양의 텍스트 데이터를 학습하여 인간의 언어를 처리하고,
  질문에 대한 답변이나 새로운 텍스트를 생성할 수 있는 인공지능 모델

- LLM은 학습 과정에서 언어의 구조와 문맥적 관계를 파악하며,
  이를 바탕으로 질의응답, 번역, 요약, 글쓰기 등 다양한 언어 관련 작업을 수행함

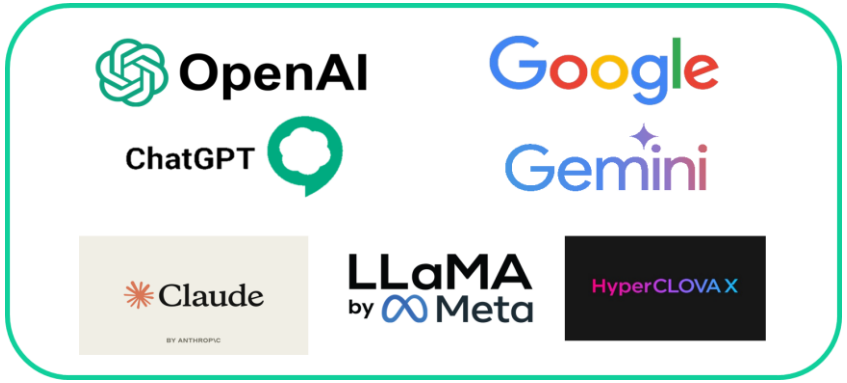
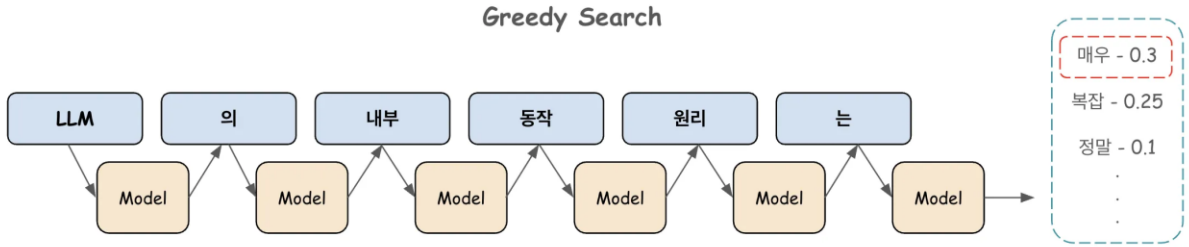


image source: https://gagadi.tistory.com/59

LLM의 작동 원리



LLM의 대표적인 텍스트 생성 방식?

- 앞서 주어진 문맥을 바탕으로 다음에 올 언어 요소를 예측하는 것

LLM의 작동 원리 - 계속


입력 문장
"대한민국의 수도는 ___"
문맥을 분석하여 다음 표현 예측
서울
90%
부산
7%
인천
3%
설명을 위한 가상의 확률 예시로, 실제 LLM의 예측 결과는 아님
생성 결과
"대한민국의 수도는 서울입니다."

LLM의 작동 원리 - 계속

LLM의 대표적인 텍스트 생성 방식?

- 앞서 주어진 문맥을 바탕으로 다음에 올 언어 요소를 예측하는 것

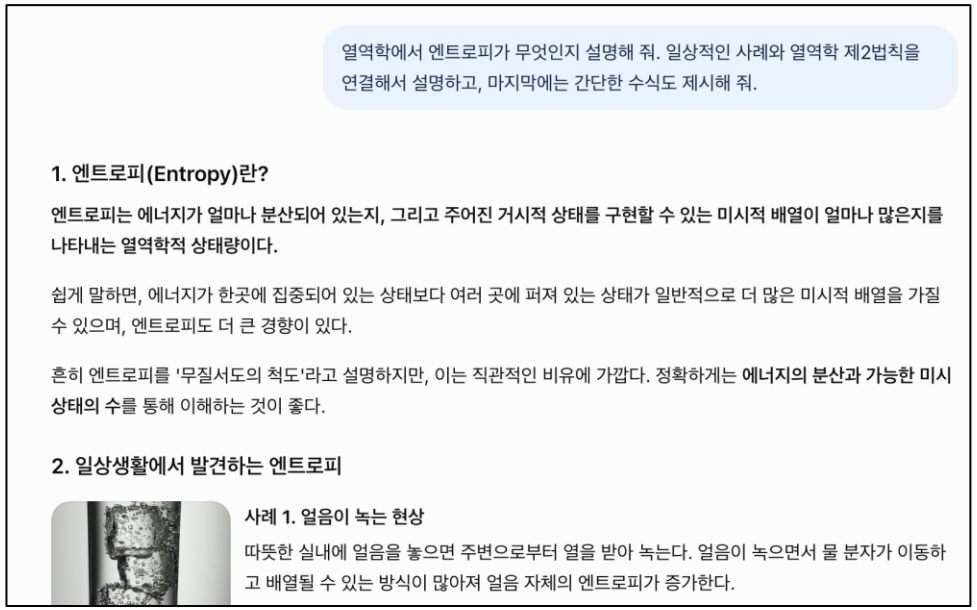
- 실제 LLM은 단어 전체가 아닌 토큰(Token) 이라는
  텍스트 처리 단위를 사용하기도 하며, 이러한 예측을 반복하여 문장을 완성

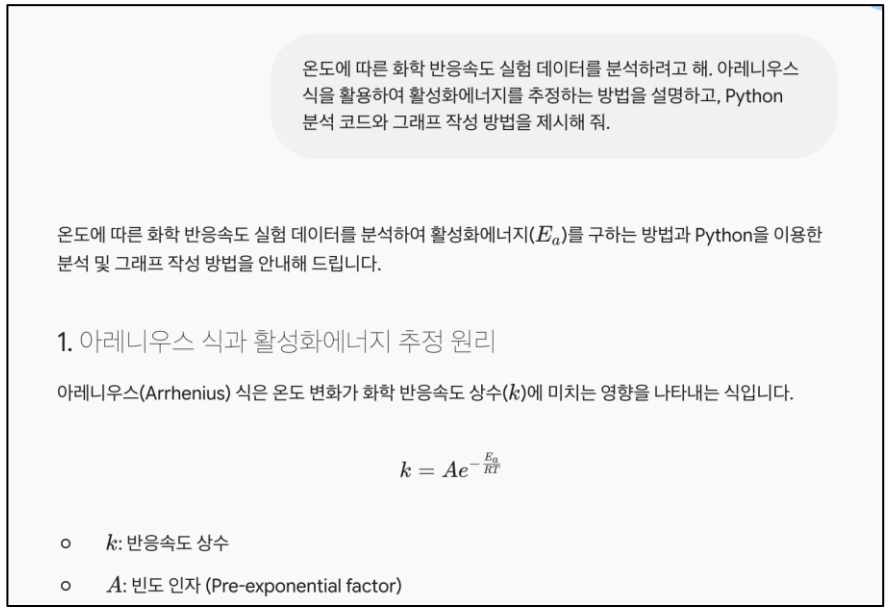
- 현대의 많은 LLM은 문장 내 요소들의 관계를 효과적으로 처리하는
  트랜스포머(Transformer) 기술을 활용

image source: https://tech.kakaopay.com/post/how-llm-works/

대학생을 위한 LLM 서비스의 대표적인 활용 예시

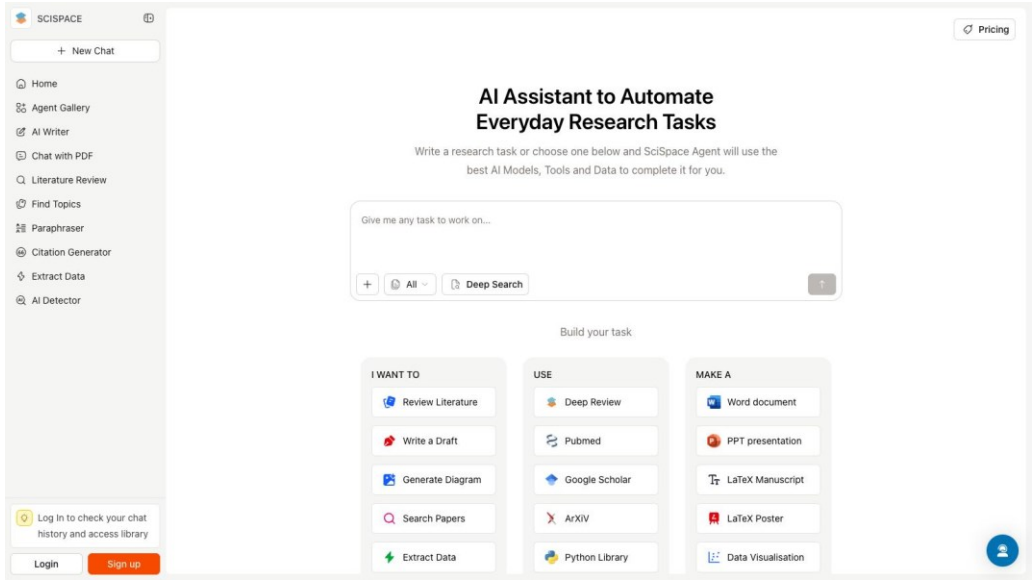
- ChatGPT, Gemini, Claude와 같은 대표적인 LLM 기반 서비스는 전공 생활에서의 학습과 연구 과정에서 개념 이해, 문헌 탐색, 데이터 분석, 프로그래밍, 보고서 분석 등 다양한 활동을 지원할 수 있음

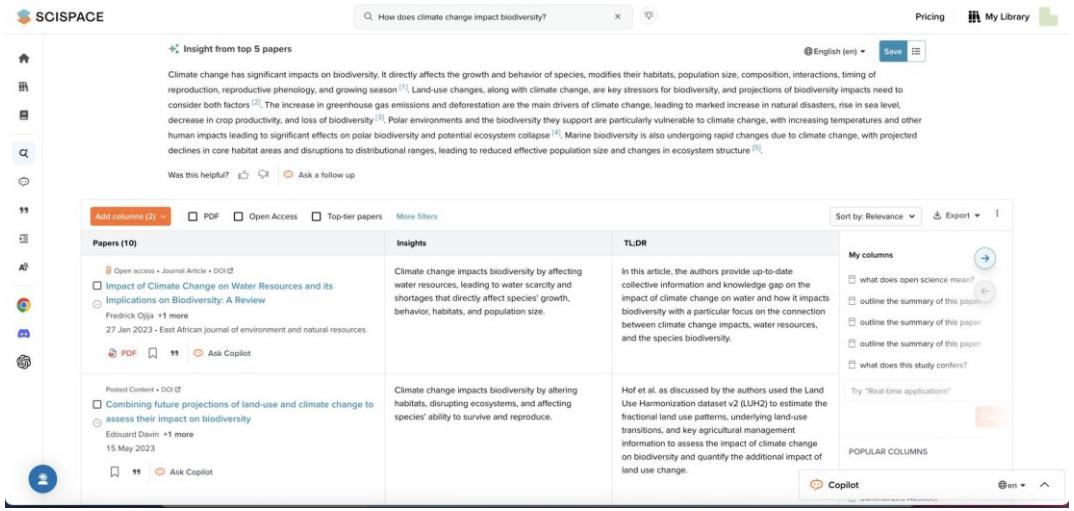




대학생을 위한 LLM 서비스의 활용 예시 - SciSpace

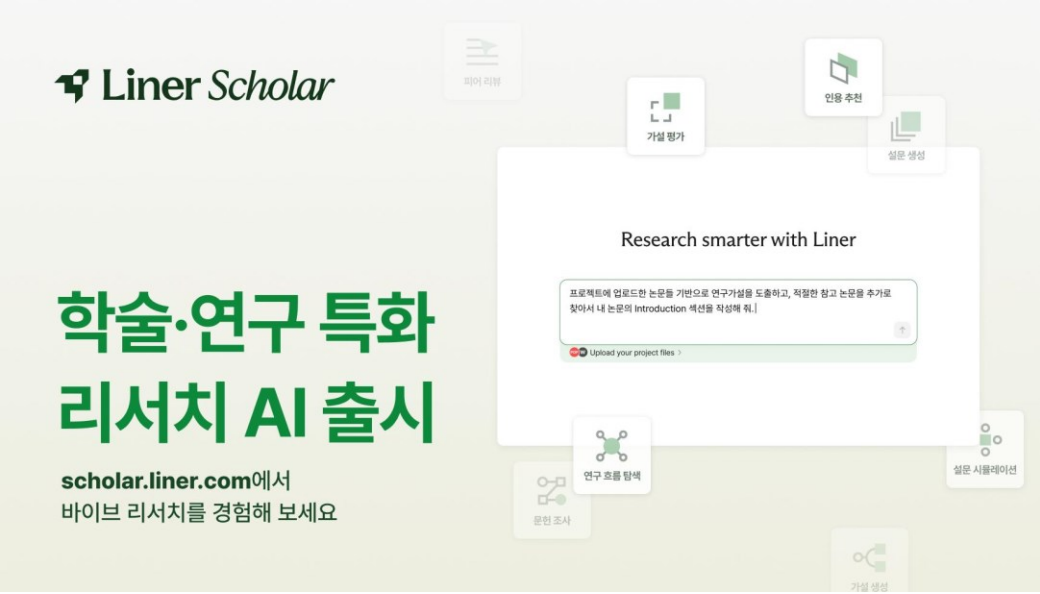
- 과학 논문을 찾고, 읽고, 분석하고, 작성하는 과정을 AI로 지원하는 연구 플랫폼
- 논문 검색뿐 아니라 문헌 리뷰, PDF 질의응답, 데이터 추출, 인용 생성, 학술 글쓰기 등을 지원

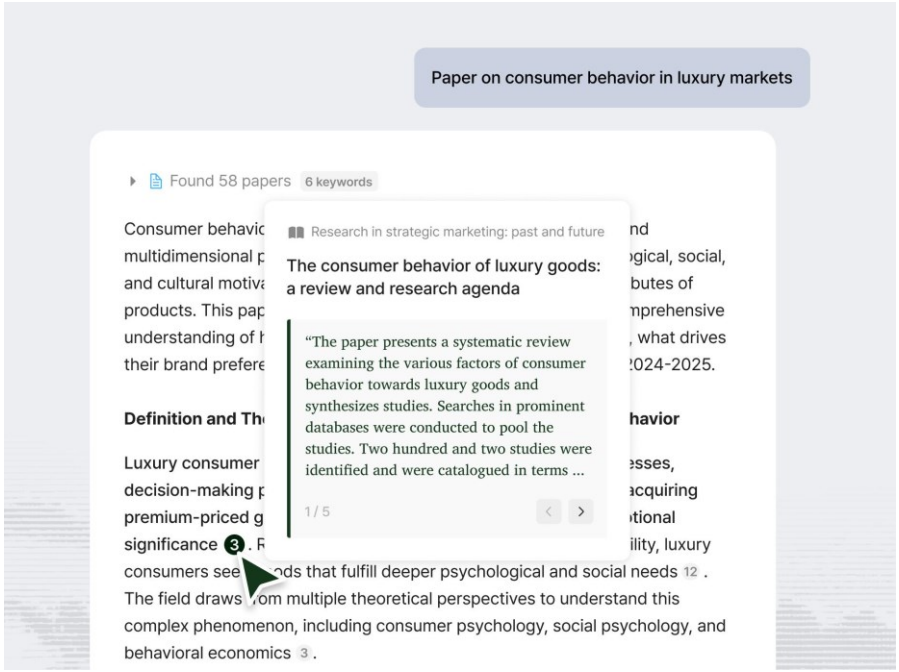




대학생을 위한 LLM 서비스의 활용 예시 - 라이너 (LINER)

- 한국의 AI 검색/리서치 플랫폼으로 웹과 학술 자료 탐색, 답변과 근거 출처 함께 제시
- 연구 특화 (Liner Scholar), 글쓰기 도구 (Liner Write)까지 확장해
  지식 업무 전반을 지원





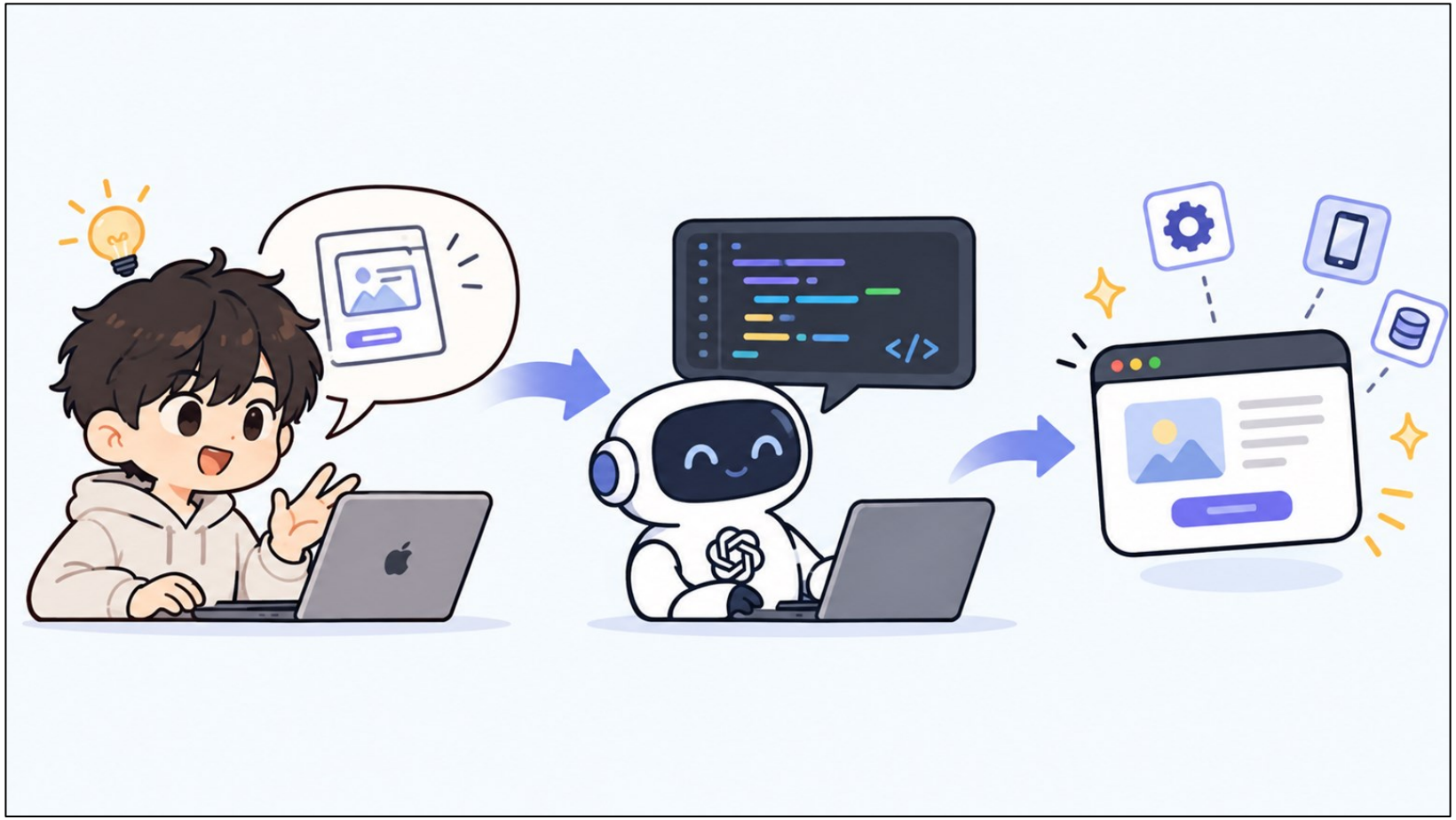
LLM의 한계

- LLM은 자연스러운 답변을 생성하지만, 모든 답변이 정확한 것은 아님

- 환각(Hallucination): 사실이 아닌 내용을 그럴듯하게 생성할 수 있음
- 편향(Bias): 학습 데이터에 포함된 편향이 답변에 반영될 수 있음
- 정보의 최신성: 외부 검색이나 최신 데이터에 연결되지 않은 경우 최근 정보를 정확히 반영하지 못할 수 있음

- 따라서 LLM이 생성한 정보는 중요한 의사결정이나 학술적 활용에 앞서 검증해야 함

바이브 코딩(Vibe Coding)이란?



바이브 코딩(Vibe Coding)이란? - 계속

- 사용자가 프로그래밍 코드를 직접 작성하는 대신, 자연어(우리가 주로 쓰는 언어)로 원하는 기능을 설명하면 생성형 AI가 코드를 작성하고 수정하도록 하는 소프트웨어 개발 방식

- 기존 프로그래밍이 코드를 작성하는 방법에 중점을 두었다면, 바이브 코딩은 무엇을 만들 것인지 정의하고 AI가 생성한 결과를 확인하는 과정에 중점을 둠

바이브코딩의 필요성

① 프로그래밍 진입장벽 완화
프로그래밍 언어와 문법에 익숙하지 않은 사람도
자연어로 원하는 기능을 설명하여 프로그램 제작을 시도할 수 있음

② 개발 시간과 반복 작업 감소
코드 작성, 수정, 기본 기능 구현을 AI가 보조하여
단순한 프로그램이나 초기 시제품을 개발하는 시간을 단축할 수 있음

바이브코딩의 필요성 - 계속

③ 아이디어의 빠른 구현과 검증
새로운 아이디어를 실제로 작동하는 프로그램이나 시제품으로 구현하고,
결과를 확인하면서 기능을 개선할 수 있음

④ 전공 지식의 실질적 활용 확대
자신의 전공 지식을 바탕으로 실험 데이터 분석 도구, 시뮬레이션,
자동화 프로그램 등을 직접 제작하는 데 활용할 수 있음

바이브코딩의 필요성 - 계속

⑤ 문제 해결 중심의 역량 강화

코드 문법을 직접 작성하는 부담을 줄여
문제 정의, 요구사항 설계, 결과 분석 및 검증에 더 많은 노력을 집중할 수 있음

기존 프로그래밍 vs. 바이브코딩

구분	기존 프로그래밍	바이브코딩
개발 방식	직접 코드 작성	자연어로 AI에 개발 요청
주요 역량	프로그래밍 언어 및 구현 능력	요구사항 정의 및 결과 검증
진입 장벽	상대적으로 높음	초기 진입장벽이 낮음
개발 속도	직접 구현에 시간 소요	간단한 시제품을 빠르게 생성 가능
결과 검증	개발자가 직접 수행	사용자 또는 개발자가 검증해야 함

바이브코딩을 할 줄 알면 코딩을 배우지 않아도 될까?

- 바이브 코딩을 활용하면 프로그래밍 언어를 직접 배우지 않고도 프로그램을 개발할 수 있기 때문에 코딩 교육이 불필요하지 않을까?

- 그러나 프로그래밍 언어와 코딩에 대한 기초 지식을 갖추고 있다면 AI에게 더욱 구체적이고 명확한 지시를 내릴 수 있으며, 생성된 코드의 오류를 스스로 판단하고 수정할 수도 있음.

- 특히 간단한 오류는 AI에 반복적으로 수정을 요청하기보다 직접 해결하는 것이 더 효율적일 수 있음.

수고하셨습니다.

AI가 척척 만들어 주는
나만의 웹사이트를 기획하려면?

AI와 함께 만드는 나만의 전공 웹 프로젝트

2차시

바이브코딩의 입력, 프롬포트(prompt)

- 생성형 인공지능이 특정 작업을 수행하거나 원하는 결과를 생성하도록 사용자가 입력하는 질문, 명령 또는 요청

- 프롬프트는 단순한 질문부터 구체적인 작업 지시까지 다양한 형태로 작성할 수 있으며, 입력 내용에 따라 AI가 생성하는 결과가 달라질 수 있음

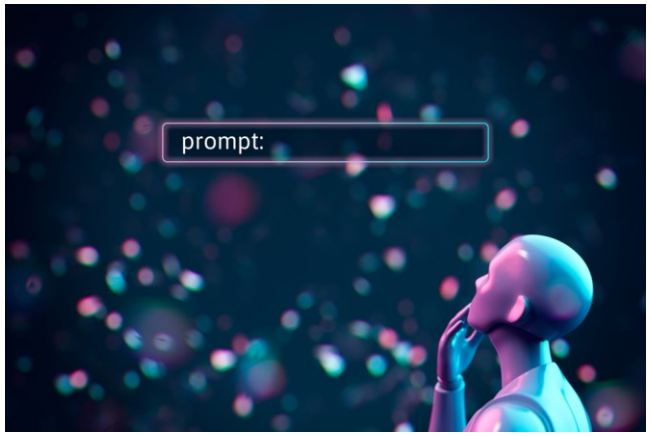




image source: https://www.shutterstock.com, https://www.booksr.co.kr/

좋은 프롬포트는 좋은 결과를 만든다

- 같은 주제를 질문하더라도 프롬프트를 어떻게 작성하느냐에 따라 답변의 구체성과 설명 수준이 달라질 수 있음

- "엔트로피가 뭐야?" vs. "열역학에서 엔트로피가 무엇인지 대학생 수준으로 설명해 줘. 일상적인 사례와 열역학 제2법칙을 연결해서 설명하고, 마지막에는 관련 수식을 제시해 줘."

좋은 프롬포트를 만들기 위해서 필요한 조건

조건	설명	예시
① 명확한 작업 지시	수행할 작업을 구체적으로 제시	엔트로피의 개념을 설명해 줘.
② 배경 및 맥락 제공	질문의 목적과 상황을 설명	열역학 입문 강의를 수강하고 있어.
③ 대상 및 수준 설정	활용 대상과 이해 수준 지정	이공계 비전공자 수준으로 설명해 줘.
④ 출력 형식 지정	답변의 구조와 표현 방식 설정	정의, 특징, 사례를 표로 정리해 줘.
⑤ 조건 및 제약 설정	분량과 범위 등의 제한 명시	500자 이내로 작성해 줘.
⑥ 근거 및 출처 명시	신뢰할 수 있는 출처 요구	정부기관 및 학술논문을 참고해 줘.

바이브 코딩을 위한 좋은 프롬포트는?

- 바이브 코딩에서는 일반적인 LLM 프롬프트와 달리, AI가 구현해야 할 기능과 프로그램의 동작 방식, 기술적 요구사항을 명확하게 전달하는 것이 중요

- 프로그램을 개발하는 목적과 목표
- 프로그램의 주 사용 대상
- 화면 구성 및 화면에 보여야하는 것

바이브코딩을 위한 좋은 프롬포트는? - 계속

- 일반적인 프롬프트는 어떤 답변을 원하는가에 초점을 맞추지만,
  바이브 코딩에서는 어떤 프로그램을 만들고, 어떻게 동작해야 하는가에 초점을 맞춤

- 또한 처음부터 모든 요구사항을 입력하기보다는 핵심 기능을 먼저 구현한 다음, 실행 결과를 확인하면서 기능을 추가하고 오류를 수정하는 반복적인 개발 방식이 효과적

마크다운(Markdown)이란?



확장자	아이콘 예시	주요 특징 / 용도
.hwp 한글 문서		• 한글(Hancom)에서 사용하는 문서 형식 • 국내에서 주로 사용 • 보고서, 과제, 공문서 등에 많이 사용
.pdf 문서 파일		• 문서의 형식을 그대로 유지하는 파일 형식 • 다양한 기기에서 동일하게 열람 가능 • 보고서, 논문, 공식 문서 배포에 주로 사용
.md 마크다운 문서		• 마크다운(Markdown) 문서 형식 • 간단한 문법으로 글을 작성할 수 있음 • 개발 문서, 노트, README 등에 주로 사용

- 복잡한 편집 도구 없이 간단한 기호와 문법을 사용하여 텍스트의 구조와 서식을 표현할 수 있는 경량 마크업 언어

- #, *, - 등의 기호를 이용하여 제목, 강조, 목록, 링크 등을 표현하며, 작성된 텍스트를 웹페이지나 문서 형태로 변환할 수 있음

마크다운(Markdown)이란? - 계속

마크다운 입력

원본 텍스트

```
# 실험 보고서

## 실험 목적

**온도**에 따른 반응속도 분석

- 데이터 수집
- 결과 분석
```

변환된 문서

실험 보고서

실험 목적

온도에 따른 반응속도 분석

- 데이터 수집
- 결과 분석

마크다운 문법을 지원하는 프로그램이 기호를 해석하여 서식을 적용한 결과를 보여준다.

바이브 코딩 요구사항 마크다운으로 정리하기

- 바이브 코딩을 통해 개발하려는 서비스 → 기능과 요구사항이 복잡하고 다양
- 따라서, 이를 체계적으로 정리하는 과정이 필요

- LLM 서비스를 이용하여 대화하며 서비스의 목적, 주요 기능, 화면 구성, 개발 조건 등을 구체화하고, 이를 마크다운 형식의 요구사항 문서로 정리한다면?

- 이렇게 작성한 마크다운 파일을 바이브 코딩 프롬포트로 제공하면 AI가 개발 목표와 요구사항을 보다 명확하게 파악할 수 있으며, 개발 과정에서 발생할 수 있는 기능 누락이나 요구사항의 불일치를 줄이는 데 도움이 됨

[프로젝트 1 소개] 나를 소개하는 포트폴리오 웹사이트

- 본인을 소개하는 포트폴리오 용도의 웹사이트를 만들어봅시다.

- 웹사이트에는 개인 인적 사항, 학과나 학점과 같은 학력 사항,
  전공 수업 시간 및 개인 시간에 수행한 프로젝트 등이 들어가면 좋을 것 같습니다.

- 각자 어필하고 싶은 포인트를 하나 정해보고,
  이를 추후 바이브 코딩 실습 수업에 만들어봅시다.

- 그렇다면 가장 먼저 해야 할 것은? LLM 서비스와 대화하며 마크다운 파일 만들기 !

수고하셨습니다.

버튼 하나로 달라지는 웹사이트의 비밀

AI와 함께 만드는 나만의 전공 웹 프로젝트

3차시

웹사이트의 주요 구성요소



웹사이트의 주요 구성요소 - 계속

- 헤더(Header): 웹사이트 상단에 위치하며 로고, 사이트명, 로그인 버튼 등을 표시하는 영역

- 내비게이션(Navigation): 사용자가 원하는 페이지로 이동할 수 있도록 메뉴와 링크를 제공하는 영역으로, 일반적으로 헤더 내부나 하단에 배치됨

- 푸터(Footer): 웹사이트 하단에 위치하며 정보, 연락처, 이용약관, 저작권 등의 정보를 제공하는 영역

프론트엔드와 백엔드

- 프론트엔드 : 사용자가 웹사이트에 접속했을 때 직접 보고 상호작용하는 영역
- 화면 구성, 디자인, 버튼, 메뉴, 사용자 입력 등의 기능을 담당

- 백엔드 : 사용자에게 직접 보이지 않는 영역
- 데이터 저장 및 관리, 사용자 인증, 서버 운영 등 웹사이트의 내부 기능을 담당

프론트엔드와 백엔드 - 계속

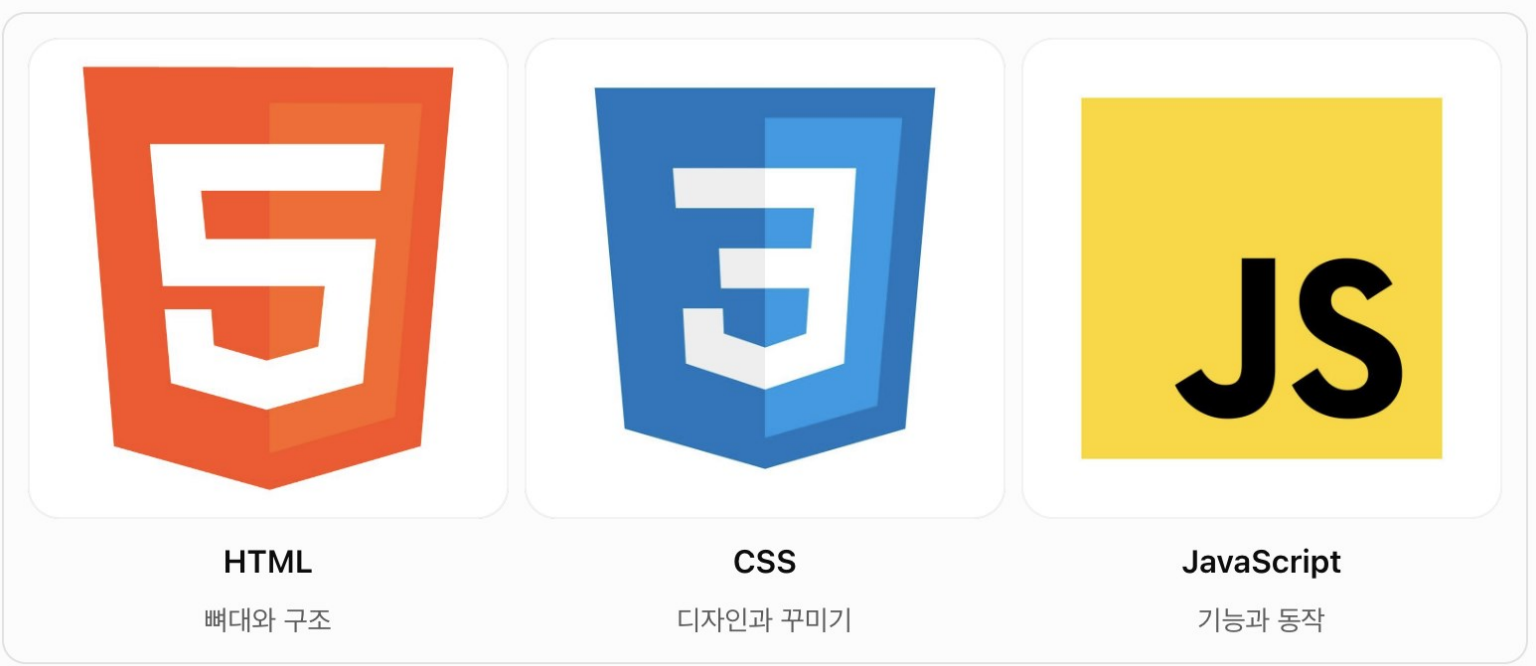
- 보통의 바이브 코딩은 프론트엔드 개발에 해당

- 인공지능을 활용하여 웹사이트의 화면을 구성하고, 버튼이나 메뉴 등의 기능을 구현하며, 사용자의 요청에 따라 디자인과 동작을 수정하는 과정을 경험하기 때문

- 이를 통해 자신의 아이디어를 실제 웹사이트 형태로 구현할 수는 있지만,
  로그인, 서버 기반 데이터 저장, 여러 사용자 간 정보 공유 등의 기능은
  일반적으로 백엔드 개발이 추가로 필요

HTML + CSS + JS = Website !

- 웹사이트의 프론트엔드는 기본적으로 HTML, CSS, JavaScript라는 세 가지 웹 기술을 활용하여 개발



HTML (HyperText Markup Language)

1. HTML (HyperText Markup Language)

- HTML은 웹페이지의 기본 구조와 콘텐츠를 정의하는 마크업 언어
- 제목, 문단, 이미지, 버튼, 링크 등의 요소를 배치하고 각 요소의 의미와 역할을 지정
- 건축물에 비유하면 건물의 뼈대와 공간 구조를 만드는 역할

CSS (Cascading Style Sheets)

2. CSS (Cascading Style Sheets)

- CSS는 HTML로 구성된 웹페이지의 디자인과 시각적 표현을 담당하는 스타일 시트 언어
- 글꼴, 색상, 크기, 여백, 배치 등을 설정하며, 화면 크기에 따라 레이아웃이 달라지는 반응형 웹 디자인에도 활용
- 건축물에 비유하면 벽지, 색상, 인테리어 등 외관을 꾸미는 역할

JavaScript (JS)

3. JavaScript (JS)

- JavaScript는 웹페이지에 동적인 기능과 사용자와의 상호작용을 구현하는 프로그래밍 언어
- 버튼 클릭, 사용자 입력 처리, 실시간 화면 변경, 서버와의 데이터 통신 등을 수행할 수 있음
- 건축물에 비유하면 조명 스위치, 자동문, 엘리베이터 등 실제로 작동하는 기능을 구현하는 역할을 한다.

바이브 코딩과 프로그래밍 기초 지식

- 바이브 코딩에서는 사용자가 개발 언어나 기술을 별도로 지정하지 않을 경우, AI가 개발 환경과 요구사항에 따라 적절한 기술을 선택하여 프로그램을 구현함

- 웹사이트 개발에서는 주로 HTML, CSS, JavaScript를 기반으로 하며, React와 TypeScript 등의 기술이 함께 활용되기도 함

바이브 코딩과 프로그래밍 기초 지식 - 계속

- 따라서 앞서 학습한 웹사이트의 구성 요소와 기본적인 웹 개발 기술을 이해하고 있다면, AI에게 사용할 기술을 직접 지정하거나 원하는 기능을 더욱 구체적이고 명확하게 요청할 수 있음

- 또한 생성된 코드의 구조와 각 요소의 역할을 이해한다면, 간단한 오류나 디자인을 직접 수정할 수 있어 AI에 반복적으로 수정을 요청하는 것보다 효율적으로 개발할 수 있음

수고하셨습니다.

내 웹사이트,
안전하게 사용하는 꿀팁

AI와 함께 만드는 나만의 전공 웹 프로젝트

4차시

바이브코딩으로 만든 웹사이트, 안전할까?

- 바이브 코딩을 활용하면 프로그래밍에 대한 전문적인 지식이 부족하더라도 웹사이트를 빠르게 개발할 수 있지만, AI가 생성한 코드의 안전성이 항상 보장되는 것은 아님

- AI가 작성한 코드에는 보안 취약점이나 잘못된 접근 권한 설정 등이 포함될 수 있으며, 이로 인해 개인정보 유출이나 외부 공격 등의 문제가 발생할 수 있음

- 특히 로그인, 회원가입, 개인정보 저장, 결제 기능 등이 포함된 웹사이트는 사용자의 중요한 정보를 처리하므로 더욱 철저한 보안 검토가 필요함.

  (다만, 이번 실습은 별도의 로그인이나 개인정보 저장 기능이 없는 프론트엔드 중심 웹사이트를 제작할 예정이기 때문에 상대적으로 위험을 제한하기 쉽습니다.)

API(Application Programming Interface)

- API는 서로 다른 소프트웨어나 시스템이 정해진 규칙에 따라 데이터를 주고받거나 기능을 사용할 수 있도록 연결하는 인터페이스

- API를 활용하면 다른 서비스의 기능을 처음부터 직접 개발하지 않고도 자신의 프로그램에 연동하여 사용할 수 있음

API(Application Programming Interface) - 계속



바이브 코딩에서의 API의 활용

- 바이브 코딩에서는 API를 활용하여 직접 개발하기 어려운 외부 서비스의 기능이나 데이터를 웹사이트에 연동할 수 있음

- 예를 들어 날씨 정보를 제공하는 웹사이트를 개발할 때, 기상 데이터를 직접 수집하는 시스템을 구축하지 않고도 날씨 API를 통해 필요한 정보를 가져올 수 있음. 마찬가지로 지도, 번역, 생성형 AI 등의 기능도 관련 API를 활용하여 구현할 수 있다.

- 따라서 API의 개념과 역할을 이해하고 있다면 AI에게 외부 서비스 연동을 구체적으로 요청할 수 있으며, 바이브 코딩으로 개발할 수 있는 웹사이트의 기능을 더욱 확장할 수 있음

API키와 보안

- 정리하자면, API 키는 외부 서비스의 API를 이용하기 위해 발급받는 고유한 인증 정보로, 서비스 이용 권한을 확인하고 사용량을 관리하는 데 활용됨

- API 키는 비밀번호와 같이 보호해야 하는 중요한 정보이며, 외부에 유출될 경우 제3자가 해당 키를 이용하여 서비스를 무단으로 사용하거나, 접근 권한에 따라 중요 정보에 접근할 위험이 있음

- 특히 일부 API는 사용량에 따라 요금이 부과되므로, API 키가 유출되면 본인이 사용하지 않은 API 호출에 대해서도 예상하지 못한 비용이 발생할 수 있음

API키와 보안 - 계속

- 바이브 코딩에서는 AI가 생성한 코드에 API 키가 그대로 포함될 수 있으므로, 비밀 API 키를 웹사이트의 프론트엔드 코드에 직접 작성하거나 GitHub와 같은 공개 저장소에 업로드하지 않도록 주의해야 함

- API 키는 서버의 환경 변수나 보안 저장소에 안전하게 보관하고, API 사용량을 주기적으로 확인하며, 가능한 경우 접근 권한과 사용 한도를 제한하는 것이 중요함



image source: https://ai.google.dev/gemini-api/docs/api-key?hl=ko

서버(Server)와 데이터베이스(Database)

서버(Server)

- 사용자의 요청을 받아 필요한 작업을 처리하고 결과를 제공하는 시스템.
- 로그인 및 회원가입 처리, 데이터 조회, 외부 API 연동 등의 기능을 담당

데이터베이스(Database)

- 서비스 운영에 필요한 정보를 체계적으로 저장하고 관리하는 시스템
- 사용자 계정 정보, 게시글, 학습 기록 등을 저장하고
  필요할 때 조회하거나 수정하는 데 활용됨

서버(Server)와 데이터베이스(Database) - 계속

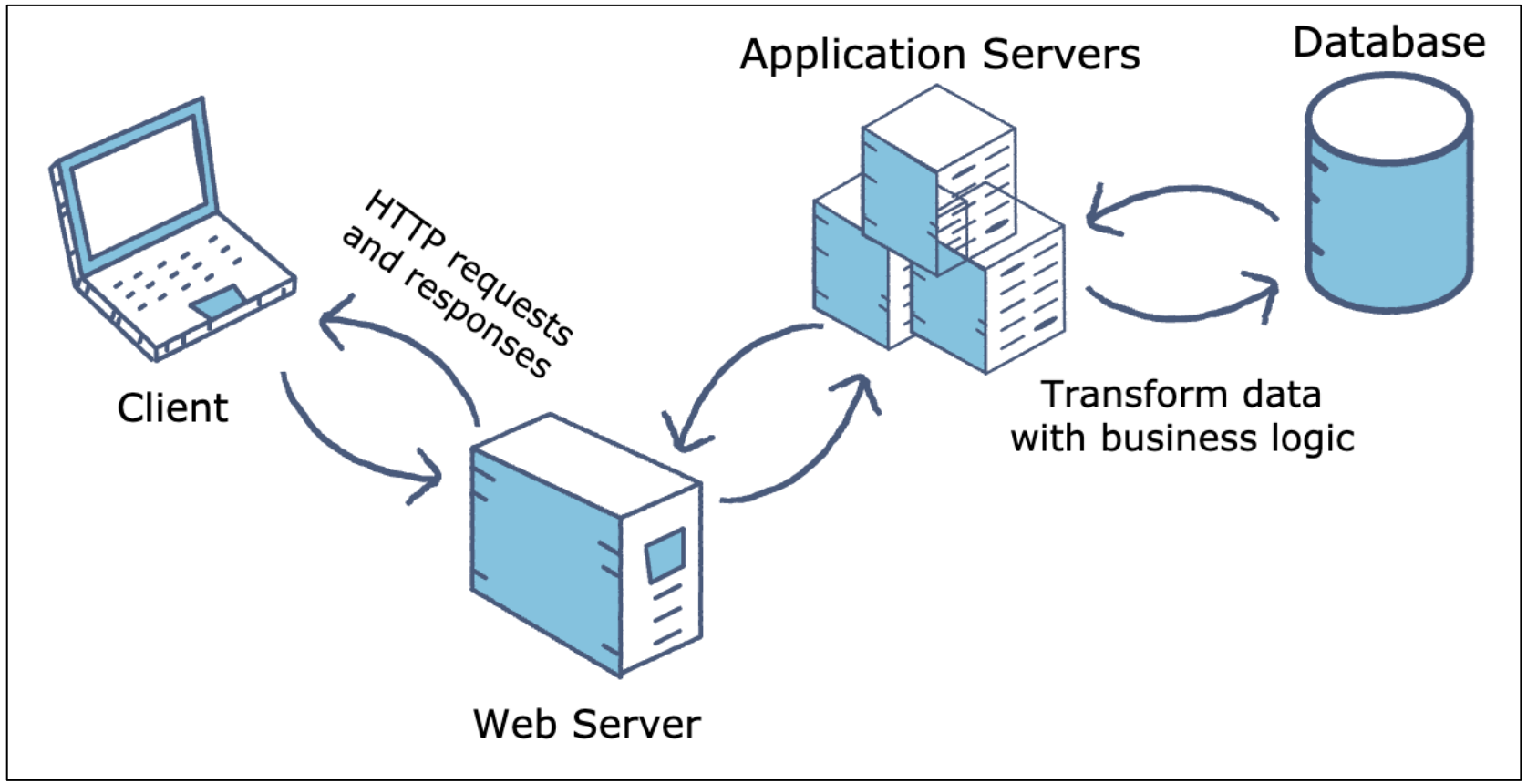


image source: https://iknow-dev.tistory.com/28

서버(Server)와 데이터베이스(Database) - 계속

- 서버와 데이터베이스를 활용하면 단순히 화면을 보여주는 웹사이트를 넘어 회원 관리, 데이터 저장, 사용자별 맞춤 기능 등을 제공하는 웹서비스를 개발할 수 있음

- 다만 서버와 데이터베이스를 활용하기 위해서는 별도의 개발 환경 구성과 서비스 연동이 필요하며, 사용자 인증, 접근 권한 설정, 개인정보 보호 등 추가적인 보안 관리가 요구됨

- 따라서 이번 바이브 코딩 실습에서는 프론트엔드 개발에 집중하며, 제한된 실습 시간과 개발 환경의 복잡성 및 보안 문제를 고려하여 서버 구축과 데이터베이스 연동은 실습 범위에서 제외함

수고하셨습니다.

내가 만든 사이트에 인터넷 주소 만들기

AI와 함께 만드는 나만의 전공 웹 프로젝트

5차시

오늘의 학습 목표

GitHub와 Vercel의 역할을 구분한다.

HTML 파일을 저장소에 올리고 인터넷 주소로 접속한다.

CLI의 의미와 활용 이유를 설명한다.

파일 수정 후 실제 사이트의 변경까지 확인한다.

오늘 배포할 사이트



파도 사이: 코스 선택과 저장 기능이 있는 가상 여행 매거진

내 컴퓨터의 파일과 인터넷 사이트

내 컴퓨터의 HTML

file:///…/index.html

내 컴퓨터의 파일을 직접 연다.

경로만 보내면 다른 사람이 열 수 없다.

배포한 사이트

https://…vercel.app

배포 서비스가 파일을 제공한다.

공개 주소로 다른 사람도 접속한다.

배포는 방문자가 접속할 수 있도록 사이트를 인터넷에 제공하는 과정

GitHub와 Vercel의 역할

GitHub

프로젝트 파일 보관

커밋으로 변경 이력 관리

코드와 변경 내용을 확인하는 곳

Vercel

연결된 저장소의 파일로 사이트 배포

배포 상태와 사이트 주소 관리

방문자가 사용할 화면을 제공하는 곳

이번 수업에서는 GitHub에 저장하고 Vercel로 배포한다.

저장소와 변경 이력

저장소 Repository
프로젝트의 파일과 변경 기록을 모아두는 공간

커밋 Commit
프로젝트의 변경 상태를 기록하는 단위

커밋 메시지
무엇을 바꿨는지 설명하는 짧은 문장

예시
첫 웹페이지 업로드 / 제목과 소개 문구 수정

오늘의 배포 순서

1 파일 확인
제공된 index.html을 브라우저에서 실행

2 GitHub에 저장
저장소 생성 후 파일 업로드와 커밋

3 Vercel로 배포
저장소 연결 후 운영 사이트 주소 확인

4 업데이트 확인
문구 수정과 커밋 후 새 배포 결과 확인

실습 준비

이메일 인증이 가능한 GitHub 계정

GitHub와 연결할 Vercel 개인 계정

압축을 푼 index.html 파일 하나

공개 가능한 예제만 사용하고 비밀번호와 API 키는 제외

저장소 이름: my-first-site / 사진과 글꼴도 HTML 안에 포함

GitHub 회원가입

1
github.com/signup에 접속한다.

2
이메일 또는 제공되는 가입 방식으로 계정을 만든다.

3
화면 안내에 따라 이메일과 추가 인증을 마친다.

4
로그인 후 프로필에서 사용자 이름을 확인한다.

이미 계정이 있다면 같은 계정으로 로그인해서 진행

GitHub 가입 화면

GitHub 저장소 생성

Owner
본인의 개인 계정

Repository name
my-first-site

Visibility
Public: 공개용 연습 자료만 사용

README
추가한 뒤 Create repository 선택

Public은 이번 수업의 선택이며 Vercel 배포의 필수 조건은 아님

HTML 파일 드래그 업로드

1
Add file에서 Upload files를 선택한다.

2
index.html 파일 하나를 업로드 영역에 끌어놓는다.

3
파일 이름과 경로가 맞는지 확인한다.

4
커밋 메시지를 입력하고 Commit changes를 완료한다.

파일을 놓는 것과 커밋 완료는 별도의 단계

업로드 후 확인

저장소 최상위에 index.html이 보인다.

파일을 열면 HTML 코드가 보인다.

첫 웹페이지 업로드 커밋이 남아 있다.

이제 Vercel에서 이 저장소를 가져와 배포한다.

CLI란?

CLI
Command Line Interface

사용 방식
글자로 명령어를 입력해 도구를 사용

터미널
명령을 입력하고 결과를 확인하는 창

오늘의 방식
웹 화면에서 클릭하고 파일을 드래그해 업로드

CLI를 배우면 도움이 되는 일

반복하는 작업을 일정한 순서로 실행

여러 파일과 개발 작업을 함께 관리

개발 도구와 AI 코딩 도구의 명령 이해

예시: git status
Git 작업 폴더의 상태를 확인하는 명령

지금은 개념만 이해하며 명령어를 실행하지 않는다.

Vercel 회원가입

1
vercel.com/signup에 접속한다.

2
Continue with GitHub를 선택한다.

3
방금 저장소를 만든 GitHub 계정으로 연결한다.

4
개인 비상업 실습은 Hobby 플랜을 기준으로 진행한다.

Vercel 가입 화면

GitHub 연결과 저장소 접근 권한

연결 화면에서 GitHub 계정 이름을 확인한다.

Vercel GitHub App 설치 또는 접근 권한을 확인한다.

저장소 선택 화면에서는 my-first-site를 허용한다.

저장소가 안 보이면 계정과 접근 권한부터 확인한다.

Vercel 로그인과 GitHub 저장소 접근 권한을 각각 확인

Vercel 프로젝트 가져오기

1
대시보드에서 새 프로젝트를 만든다.

2
Import Git Repository 영역을 확인한다.

3
소유자와 my-first-site 저장소 이름을 확인한다.

4
Import를 선택해 프로젝트 설정으로 이동한다.

HTML 한 파일에 맞는 배포 설정

Framework Preset
Other

Root Directory
저장소 최상위

Build Command
Override를 켜고 빈 값으로 유지

Output Directory
기본값 유지: public 폴더가 없으면 루트 파일 제공

이번 예제는 index.html 하나이며 별도 설치와 빌드 명령이 필요 없다.

첫 배포와 완료 상태 확인

1
저장소와 설정을 확인하고 Deploy를 선택한다.

2
배포 상태와 로그를 확인한다.

3
Ready 등 완료 상태가 되면 사이트를 연다.

4
오류가 나면 파일 위치와 배포 설정을 점검한다.

배포 버튼을 누른 뒤 완료 상태까지 확인

저장소 주소와 사이트 주소

GitHub 저장소 주소

github.com/계정/my-first-site

파일과 변경 이력을 확인

코드 제출 또는 코드 공유에 사용

Vercel 운영 사이트 주소

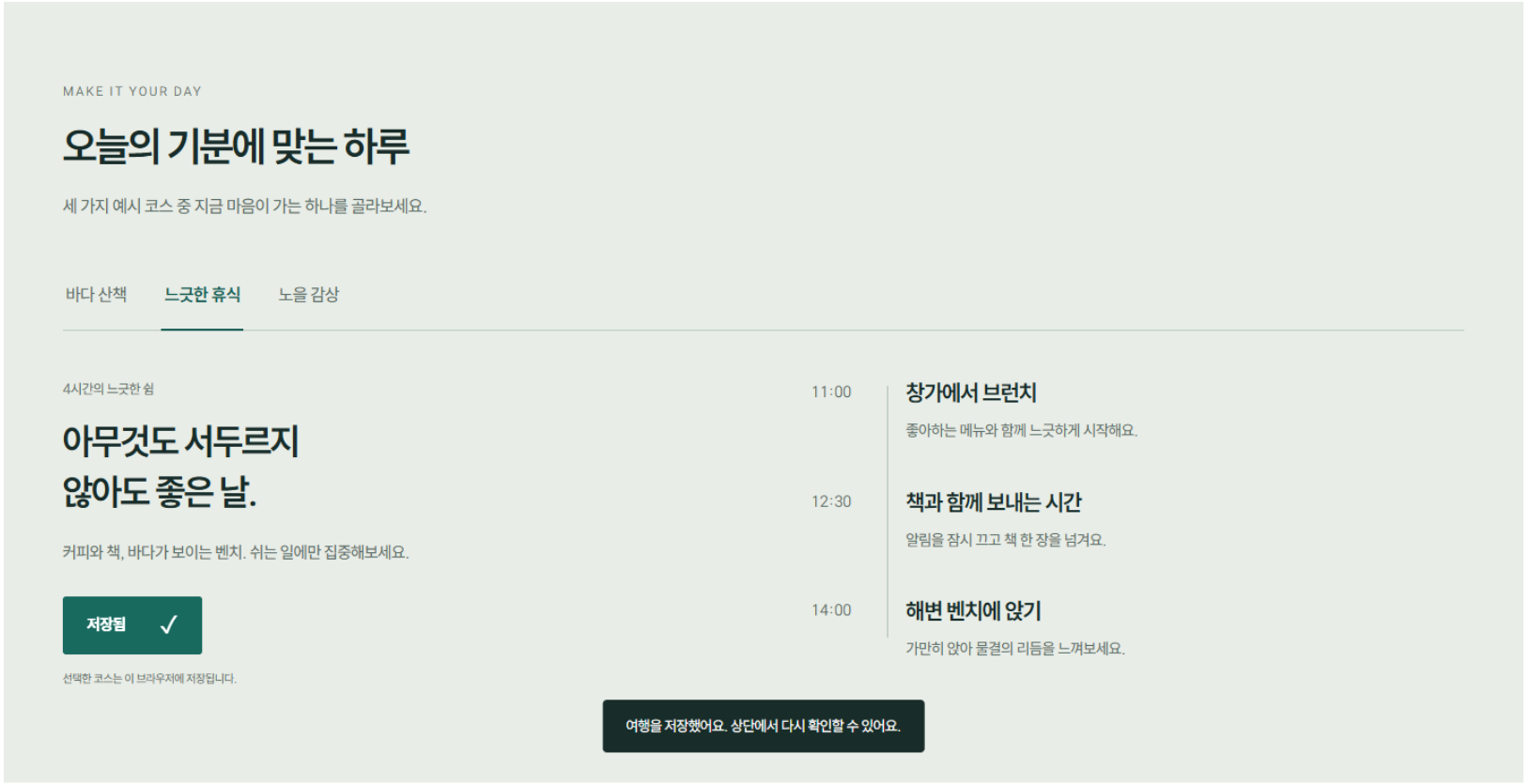
프로젝트에 연결된 Production 도메인

완성된 화면과 기능을 확인

방문자에게 공유할 때 사용

프로젝트의 운영 도메인과 배포별 생성 주소를 구분한다.

배포한 사이트 확인



운영 주소에서 코스 선택과 저장 확인, 로그인하지 않은 창에서도 접속

HTML 수정 실습

화면 제목 h1
이번 주말, 바다로 가요

소개 문구 #intro
좋아하는 풍경을 골라 나만의 하루를 완성하세요.

버튼 색상
--button-color: #155b52; → #2549a5;

파일 위쪽의 수업 수정 ① ② ③ 주석에서 세 곳만 수정

수정한 내용을 GitHub에 반영

1

index.html의 웹 편집기를 연다.

2

제목과 소개 문구, 버튼 색을 수정한다.

3

변경 비교에서 의도한 부분만 바뀌었는지 확인한다.

4

메시지를 입력하고 main 브랜치에 커밋한다.

커밋 메시지 예시: 제목과 소개 문구 및 버튼 색 수정

자동 재배포 과정 확인

1 GitHub
main에 새 커밋이 기록된다.

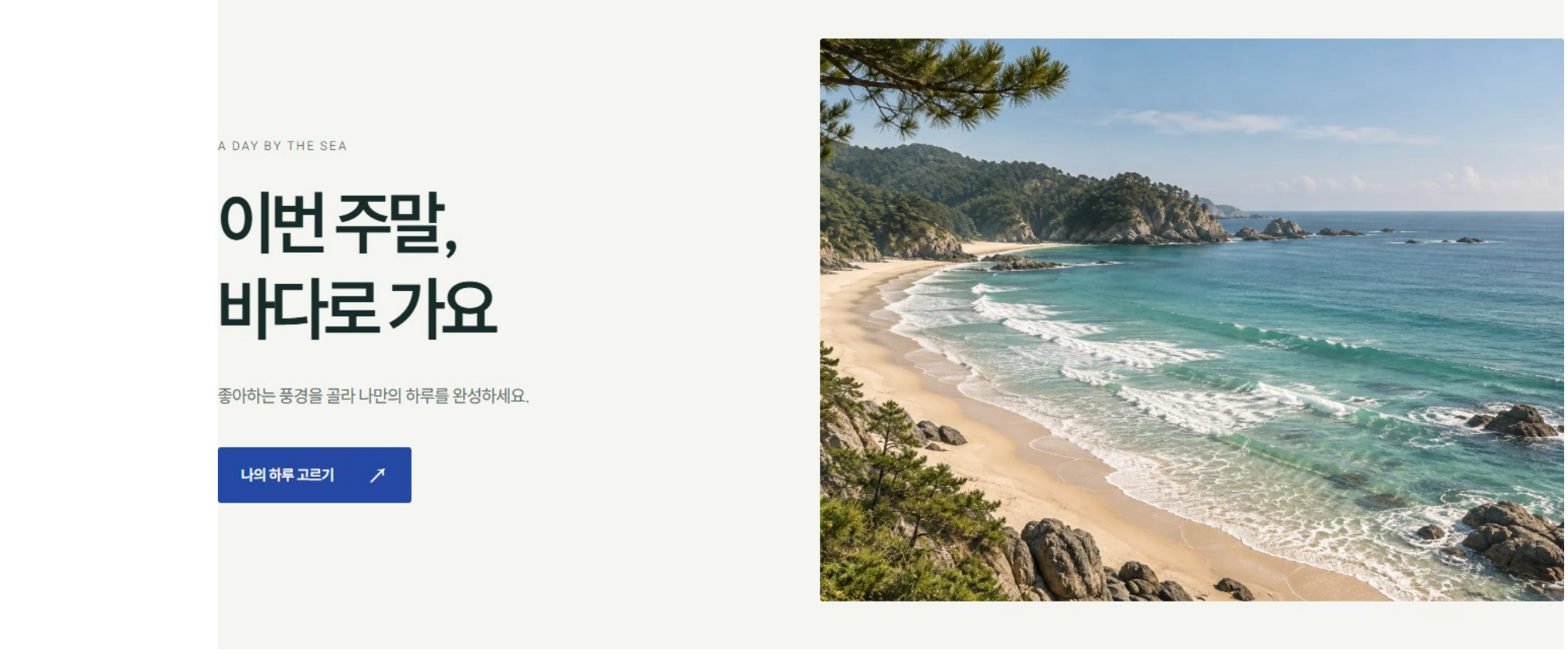
2 Vercel
연결된 프로젝트에서 새 배포가 시작된다.

3 배포 목록
커밋과 상태를 확인하고 완료까지 기다린다.

4 운영 사이트
처음 사용한 같은 주소를 새로고침한다.

자동 배포가 진행 중이면 별도의 파일 업로드를 반복하지 않는다.

같은 주소에서 수정 결과 확인



새 제목과 소개 문구, 파란색 버튼 확인 후 코스 기능도 다시 테스트

문제가 생겼을 때 확인할 것

저장소가 목록에 없음
GitHub 계정과 Vercel 앱의 저장소 접근 권한

404 화면이 나옴
index.html 이름과 위치, 루트와 출력 설정

수정이 안 보임
커밋 완료, 새 배포 상태, 운영 주소, 새로고침

다른 사람은 접속 불가
공유한 주소와 해당 프로젝트의 배포 보호 설정

학습 확인

파일과 변경 이력을 관리하는 서비스는?

방문자에게 완성된 화면을 보여줄 주소는?

파일을 수정한 뒤 사이트 확인 전에 볼 상태는?

CLI를 배우면 어떤 작업에 도움이 될까?

실습 완료 확인

저장소 최상위에 index.html이 있다.

운영 주소로 사이트가 열리고 코스 선택이 작동한다.

수정 후 같은 주소에서 새 문구와 색이 보인다.

저장소 주소와 운영 사이트 주소를 각각 기록했다.

다음 차시: AI로 나만의 전공 소개 사이트 만들기

수고하셨습니다.

나를 소개하는
포트폴리오 사이트 만들기

AI와 함께 만드는 나만의 전공 웹 프로젝트

6차시

오늘 만들 결과

기획서
2차시 MD 파일에 디자인 규칙 추가

첫 번째 사이트
Google AI Studio의 Build에서 생성

확인 기록
전공·관심 분야·활동·프로젝트 점검

다음 차시 준비
프로젝트와 수정할 내용 보관

강사 예시와 나의 포트폴리오

강사 시연 예시

홍길동 / 의공학전공 4학년

생체신호 AI · 물리 기반 모델링

심화 연구 프로젝트 소개

여러분의 포트폴리오

본인의 이름 · 전공 · 소개

각자의 관심 분야와 활동

과제 · 동아리 · 개인 프로젝트

제작 과정은 함께 따라가고, 소개할 내용은 각자의 기획서로 준비합니다.

6차시와 7차시의 연결

6차시

내용과 디자인 규칙 정리

첫 사이트 생성

기획서와 결과 비교

7차시

이미지와 요소 수정

스마트폰 화면 점검

GitHub·Vercel 배포

누가, 무엇을 읽게 할까요?

방문자
나와 함께 공부하거나 프로젝트를 할 사람

핵심 소개
어떤 분야를 공부하고 무엇에 관심이 있는가

경험의 근거
직접 한 일과 그 과정에서 배운 점

다음 행동
프로젝트 설명으로 자연스럽게 이동

소개할 내용은 네 가지

나의 소개
이름 · 전공 · 소개 문장, 학년은 선택

관심 분야
지금 배우거나 더 탐구하고 싶은 주제

활동 경험
수업 · 동아리 · 스터디 · 대외 활동

프로젝트
목적 · 내 역할 · 과정 · 결과와 배운 점

디자인 시스템이란?

사이트 전체에서 함께 사용하는 디자인 규칙

색상과 글꼴, 글자 크기에 역할을 정하기

버튼과 카드의 모양을 반복해서 사용하기

간격을 정해 화면의 흐름을 맞추기

출처: GOV.UK Design System (발표자 노트 참고)

“예쁘게”를 규칙으로 바꾸기

막연한 요청

깔끔하게 만들어 줘

제목은 눈에 띄게

버튼은 예쁘게

확인할 수 있는 요청

밝은 배경과 파란 강조색

큰 제목과 읽기 쉬운 본문

버튼 높이 48px, 모서리 8px

색상은 역할부터 정하기

배경

#F7F8FA



본문

#17212B



강조

#2855D9

파란색은 주요 버튼과 중요한 표시에서 반복해서 사용합니다.

글자는 크기와 역할을 함께

글꼴
Freesentation, 사용할 수 없으면 읽기 쉬운 산세리프

첫 제목
큰 화면 56px, 작은 화면 36px

본문
큰 화면 18px, 작은 화면 16px

줄 간격
본문 글자 크기의 약 1.7배

출처: GOV.UK Design System (발표자 노트 참고)

버튼·카드·간격의 약속

버튼
높이 48px 이상, 모서리 8px

프로젝트 카드
모서리 16px, 같은 구성과 여백

간격
8·16·24·48·80px를 기본 단계로 사용

전체 폭
최대 1120px, 좌우 여백 확보

MD 기획서에 더할 내용

기존 내용 유지
목적·대상·소개할 정보·필요한 영역

디자인 규칙 추가
색상·글자·버튼·카드·간격

동작 명시
메뉴와 버튼이 이동할 위치

완성 기준 추가
네 가지 소개 내용과 기본 동작 확인

실습 1 · 나의 소개와 경험 정리

본인의 이름·전공·소개 문장

내가 선택한 관심 분야와 활동

프로젝트 또는 과제의 목적과 내 역할

진행 과정·현재 결과·배운 점

출처: McKeen et al. / arXiv / Raissi et al. / Science (발표자 노트 참고)

실습 2 · 디자인 규칙 추가하기

디자인 규칙 블록 복사

기존 기획서 아래에 추가하거나 교체

서로 충돌하는 문장 정리

portfolio-plan.md로 저장

Google AI Studio에서 할 일

프롬프트 작성
어떤 결과가 필요한지 설명

모델 응답 시험
설명과 아이디어를 주고받기

Build로 제작
기획서를 바탕으로 사이트 생성

결과 확인
미리보기와 생성된 파일 살펴보기

출처: Google AI for Developers (발표자 노트 참고)

Build 화면에서 찾을 세 곳

요청 입력
기획서와 수정 요청을 전달하는 곳

미리보기
사용자가 보게 될 화면을 확인하는 곳

코드·파일
AI가 만든 프로젝트 구성을 살펴보는 곳

실제 메뉴 이름과 배치는 촬영 화면을 기준으로 안내합니다.

출처: Google AI for Developers (발표자 노트 참고)

실습 3 · Build 시작하기

aistudio.google.com 접속

Google 계정으로 로그인

Build의 새 프로젝트 열기

플랫폼 선택이 있다면 Web 선택

Google AI Studio 열기

출처: Google AI for Developers (발표자 노트 참고)

생성 요청에 들어갈 네 가지

작업
나의 포트폴리오 웹사이트 제작

자료
내 소개 · 관심 · 활동 · 프로젝트

규칙
내 기획서에 적은 디자인과 화면 동작

확인 기준
내 정보 · 선택한 내용 · 메뉴 동작

실습 4 · 기획서로 사이트 생성

학생용 요청문에 내 기획서 전체 넣기

대괄호 안내 문구와 빈칸 정리

본인 정보·경험·디자인 규칙 확인

Build 입력창에 붙여 넣고 생성 실행

출처: Google AI for Developers (발표자 노트 참고)

생성 후 확인할 기준

내 기획서의 이름·전공·소개가 맞는가?

내가 선택한 관심·활동이 반영되었는가?

내 프로젝트의 목적·역할·과정이 보이는가?

메뉴와 주요 버튼이 약속한 곳으로 이동하는가?

실습 5 · 첫 화면과 메뉴 확인

내 이름·전공·소개 문장 확인

기획서의 의미가 유지되는지 읽기

메뉴와 주요 버튼을 눌러 보기

페이지 끝까지 내려 전체 구성 확인

실습 6 · 전공과 관심 분야 확인

이름과 전공이 내 기획서와 같은가?

소개 문장에 나의 방향이 담겼는가?

내가 선택한 관심 분야가 들어갔는가?

요약된 설명의 뜻이 달라지지 않았는가?

실습 7 · 내 역할과 과정이 보이는가?

목적
어떤 문제를 다루거나 무엇을 만들었는가

내 역할
팀의 결과와 내가 직접 한 일 구분

진행 과정
시도한 방법 · 수정한 부분 · 확인한 내용

결과와 배운 점
현재 상태 · 만든 결과물 · 다음에 개선할 점

출처: McKeen et al. / arXiv / Raissi et al. / Science (발표자 노트 참고)

AI가 만든 문장도 사실 확인

내가 제공한 내용과 담당 역할이 유지되는가?

적지 않은 경력·수상·실적이 추가되지 않았는가?

진행 상태나 결과가 과장되지 않았는가?

연락처와 외부 링크의 실제 주소를 확인했는가?

빠진 내용은 범위를 좁혀 요청

[프로젝트 이름] 소개에
내 역할과 진행 과정을 추가해 주세요.
아래 기획서의 내용을 사용하고,
다른 영역과 디자인 규칙은 유지해 주세요.

프로젝트 이름 + 빠진 항목 + 사용할 자료 + 유지할 부분

디자인 규칙도 결과와 비교

중요한 버튼에 강조색을 사용하는가?

제목과 본문의 크기가 구분되는가?

같은 종류의 카드 모양이 일관적인가?

글과 영역 사이에 읽을 여백이 있는가?

실습 8 · 다음 차시를 위해 보관

알아보기 쉬운 프로젝트 이름 사용

프로젝트를 다시 열 수 있는지 확인

코드 전체를 ZIP으로 내려받기

기획서·확인표·ZIP을 함께 보관

출처: Google AI for Developers (발표자 노트 참고)

오늘의 완료 기준

MD 기획서에 디자인 규칙이 있다

Build 미리보기에서 내 소개가 보인다

전공·관심·활동·프로젝트를 확인했다

메뉴 동작을 확인하고 프로젝트를 보관했다

다음 시간에 바꿀 세 가지

이미지
프로젝트의 분위기를 보여 주는 이미지 추가

요소
버튼·글자·카드에서 한 가지씩 수정

확인과 배포
스마트폰 화면 점검 후 인터넷에 공개

수고하셨습니다.

클릭하고 싶어지는
멋진 사이트로 업그레이드하기

AI와 함께 만드는 나만의 전공 웹 프로젝트

7차시

6차시에서 만든 실제 사이트



사용자 제공 6차시 프로젝트의 로컬 실행 화면

출처: 사용자 제공 프로젝트 (발표자 노트 참고)

강사 예시와 나의 수정 대상

강사 시연

홍길동 / 의공학전공 4학년

연구 포트폴리오의 기존 기능 유지

강사용 요청문으로 수정

학생 실습

6차시에 만든 본인의 사이트

본인의 이미지와 수정 대상 선택

학생용 요청문을 채워서 사용

오늘의 작업 순서

이미지
주제에 맞게 생성하고 사이트에 넣기

요소 수정
버튼 · 제목 · 카드의 변화를 직접 확인

작은 화면
폭을 바꾸며 배치와 동작 점검

배포
GitHub에 코드 업로드 후 Vercel 연결

실습 1 · 기존 프로젝트 열기

AI Studio에서 6차시 프로젝트 다시 열기

현재 미리보기와 기획서 비교

주요 동작 확인 후 코드 ZIP 보관

수정 전 화면 한 장 저장

출처: Google AI for Developers / 사용자 제공 프로젝트 (발표자 노트 참고)

이번 시연에서 바꿀 부분

첫 화면
오른쪽에 연구 주제를 표현하는 이미지 추가

주요 버튼
배경을 짙은 청록색으로 변경

큰 제목
큰 화면 64px, 작은 화면 38px로 요청

프로젝트 카드
마우스를 올렸을 때 작은 움직임 추가

출처: 사용자 제공 프로젝트 (발표자 노트 참고)

이미지는 소개를 돕는 자료

넣을 위치와 용도를 먼저 정하기

전공이나 프로젝트와 연결되는 주제 선택

글자를 이미지에 넣지 않고 여백 확보

연구 결과와 분위기용 이미지 구분

이미지 요청문 구성

용도와 비율
포트폴리오 첫 화면, 가로 16:9

주제
생체신호 파형과 유체의 흐름

분위기와 색
밝은 배경, 파란색과 짙은 청록색

제외할 요소
글자 · 숫자 · 기관 로고 · 실제 환자 자료

실습 2 · 이미지 생성과 선택

Gemini의 이미지 생성 화면 열기

자신의 주제에 맞춘 요청문 입력

생성 결과의 내용과 구도 확인

선택한 이미지를 내려받아 보관

출처: Google Gemini Help (발표자 노트 참고)

실습 3 · 이미지 파일 전달

기존 Build 프로젝트로 돌아오기

입력창의 파일 첨부로 이미지 선택

첨부된 이미지 미리보기 확인

넣을 위치와 작은 화면 배치 함께 요청

출처: Google AI for Developers (발표자 노트 참고)

이미지 배치와 파일 경로

큰 화면
소개 글 왼쪽, 이미지 오른쪽

작은 화면
글과 이미지가 한 열로 배치

파일 보관
프로젝트에 포함되는 정적 이미지

확인
새로고침과 내보낸 ZIP에서도 이미지 확인

출처: Vite (발표자 노트 참고)

수정 요청의 네 가지 정보

대상
어디에 있는 어떤 요소인가

변경
색 · 크기 · 간격 · 효과 중 무엇인가

유지
문구 · 내용 · 동작 중 무엇을 남길 것인가

확인
무엇을 보면 성공인지 알 수 있는가

실습 4 · 수정할 요소 선택

미리보기의 선택·주석 기능 켜기

첫 화면의 주요 버튼 지정

선택 표시와 대상 문구 함께 확인

변경 요청을 입력하고 결과 비교

출처: Google AI for Developers (발표자 노트 참고)

실습 5 · 주요 버튼 색 변경

대상: 첫 화면의 연구 프로젝트 보기

기본색 #0F766E, 마우스 위 #115E59

흰 글자와 프로젝트 이동 유지

같은 주요 버튼의 색도 함께 확인

출처: 사용자 제공 프로젝트 (발표자 노트 참고)

실습 6 · 제목 크기 변경

첫 화면의 큰 제목 선택

큰 화면 64px, 작은 화면 38px 요청

제목 문구와 본문 크기는 유지

줄바꿈과 버튼 위치 확인

출처: 사용자 제공 프로젝트 (발표자 노트 참고)

실습 7 · 카드에 반응 추가

프로젝트 카드 영역 선택

마우스 위에서 4px 위로 이동

옅은 그림자와 약 180ms 전환

작은 화면과 기존 버튼 동작 유지

출처: 사용자 제공 프로젝트 (발표자 노트 참고)

수정 후 기존 기능 확인

전체 · 생체신호 AI · 물리 기반 모델링 필터

프로젝트 펼치기와 접기

상세 창 열기와 닫기, Esc 키

메뉴와 첫 화면 버튼의 목적지

출처: 사용자 제공 프로젝트 (발표자 노트 참고)

원하지 않는 변화가 생겼다면

바뀐 위치와 현재 결과를 기록

유지할 내용과 되돌릴 부분을 구분

필요한 원문이나 화면을 함께 전달

복원 후 핵심 기능 다시 확인

반응형은 직접 확인하는 과정

화면 폭에 따라 배치와 크기가 달라짐

글자 잘림과 가로 넘침 확인

메뉴 · 버튼 · 상세 창도 직접 사용

AI의 설명과 실제 결과를 구분

실습 8 · 폭을 바꾸어 확인

큰 화면 1440px와 중간 폭 768px

작은 화면 390px와 360px

제목 · 이미지 · 메뉴 · 카드 비교

검사 화면과 실제 휴대전화 확인

출처: Chrome for Developers (발표자 노트 참고)

작은 화면의 첫 화면 점검

첨부본: 360·390px에서 상단 가로 넘침

큰 제목의 글자와 줄이 잘리지 않는가?

이미지가 글 아래에서 자연스럽게 보이는가?

메뉴와 주요 버튼이 화면 안에 보이는가?

출처: 사용자 제공 프로젝트 (발표자 노트 참고)

작은 화면의 카드와 상세 창

필터와 버튼이 화면 안에 들어오는가?

긴 설명과 제목이 자연스럽게 줄바꿈되는가?

상세 창 안에서 끝까지 스크롤되는가?

닫기 후 원래 페이지를 사용할 수 있는가?

출처: 사용자 제공 프로젝트 (발표자 노트 참고)

반응형 보완 요청 예시

가로 360px에서 상단 이름과 메뉴가 겹칩니다.
이름은 줄바꿈하고 메뉴는 화면 안에 배치해 주세요.
큰 화면의 배치와 메뉴 이동은 유지해 주세요.

실제 증상에 맞춰 폭 · 위치 · 원하는 결과를 바꾸어 사용합니다.

실습 9 · 기획서에 변경 반영

이미지의 주제와 실제 파일 위치

주요 버튼의 색과 적용 범위

제목 크기와 카드 효과

작은 화면 규칙과 남은 작업

실습 10 · 수정한 프로젝트 내보내기

최신 변경이 반영된 상태 확인

코드 전체를 ZIP으로 내려받기

압축을 풀어 내부 파일 확인

이미지와 프로젝트 설정 파일 함께 보관

출처: Google AI for Developers (발표자 노트 참고)

이번 프로젝트의 파일 구조

package.json
필요한 패키지와 build 명령

src
화면 구성과 포트폴리오 내용

public/images
추가한 이미지가 들어갈 위치

Vite 빌드
소스를 묶어 dist 폴더 생성

출처: 사용자 제공 프로젝트 / Vite (발표자 노트 참고)

업로드할 파일 확인

프로젝트 내부를 저장소의 맨 위에 올리기

src와 실제 이미지 폴더 포함

package.json과 잠금 파일 함께 보관

node_modules · dist · 실제 .env는 제외

출처: 사용자 제공 프로젝트 / GitHub Docs (발표자 노트 참고)

설치 오류가 나면 로그부터 확인

설치 실패와 화면 오류를 구분

첫 번째 오류 문장과 패키지 이름 확인

호환되는 버전으로 수정 후 다시 빌드

검증한 최신 파일 묶음을 업로드

출처: 사용자 제공 프로젝트 (발표자 노트 참고)

실습 11 · GitHub에 프로젝트 올리기

새 저장소 또는 정한 저장소 열기

Upload files로 내부 파일과 폴더 업로드

package.json · src · 이미지 위치 확인

변경 설명을 적고 Commit changes

출처: GitHub Docs (발표자 노트 참고)

실습 12 · Vercel 배포 설정

프로젝트 연결
GitHub의 해당 저장소 Import

Framework Preset
강사 예시는 Vite

Build Command
npm run build

Output Directory
dist, Root는 package.json이 있는 위치

출처: Vercel Docs (발표자 노트 참고)

배포 완료와 접속 확인

성공 상태와 연결된 변경 기록 확인

실제 사이트 주소 열기

로그인하지 않은 창에서도 접속 확인

실패하면 로그의 첫 오류부터 확인

출처: Vercel Docs (발표자 노트 참고)

실습 13 · 실제 주소에서 최종 점검

이미지와 최신 문구가 표시되는가?

메뉴 · 필터 · 상세 보기가 동작하는가?

작은 화면에서도 읽고 누를 수 있는가?

새로고침과 외부 링크도 정상인가?

출처: Chrome for Developers / Vite (발표자 노트 참고)

수정 이후 다시 배포되는 흐름

AI Studio에서 수정한 내용 확인

최신 코드를 GitHub에 반영

연결한 배포 브랜치의 새 커밋 확인

Vercel 재배포 후 실제 주소 재확인

출처: Vercel Docs (발표자 노트 참고)

오늘의 완료 기준

본인의 사이트에 이미지 적용

선택한 요소 수정과 기존 기능 확인

작은 화면 점검과 필요한 보완

GitHub · Vercel 배포 후 실제 주소 확인

나의 포트폴리오 작업 기록

변경 이유
방문자가 읽거나 사용하기 어려웠던 점

수정 요청
선택한 요소와 구체적인 변경 내용

확인 결과
화면과 기능에서 직접 확인한 변화

최종 보관
기획서 · 코드 · 저장소 · 배포 주소

수고하셨습니다